VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA STROJNÍHO INŽENÝRSTVÍ

FACULTY OF MECHANICAL ENGINEERING

ÚSTAV MECHANIKY TĚLES, MECHATRONIKY A BIOMECHANIKY

INSTITUTE OF SOLID MECHANICS, MECHATRONICS AND BIOMECHANICS

MODULÁRNÍ SYSTÉM PRO AUTOMATIZOVANÉ ŘÍZENÍ MUZEJNÍCH EXPOZIC

MODULAR SYSTEM FOR THE AUTOMATED CONTROL OF MUSEUM EXHIBITS

BAKALÁŘSKÁ PRÁCE

BACHELOR'S THESIS

AUTOR PRÁCE Andrej Žabka

AUTHOR

VEDOUCÍ PRÁCE Ing. Michal Bastl, Ph.D.

SUPERVISOR

BRNO 2026

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ FAKULTA STROJNÍHO INŽENÝRSTVÍ

Zadání bakalářské práce

Ústav:	Ústav mechaniky těles, mechatroniky a biomechaniky
Student:	**Andrej Žabka**
Studijní program:	Mechatronika
Studijní obor:	bez specializace
Vedoucí práce:	**Ing. Michal Bastl, Ph.D.**
Akademický rok:	2025/26

Ředitel ústavu Vám v souladu se zákonem č.111/1998 o vysokých školách a se Studijním a zkušebním řádem VUT v Brně určuje následující téma bakalářské práce:

Modulární systém pro automatizované řízení muzejních expozic

Stručná charakteristika problematiky úkolu:

Student se bude zabývat návrhem a realizací modulárního systému pro automatizované řízení muzejních expozic. Úkolem je vytvořit řešení, které umožní spouštění časovaných scén v jedné místnosti a zajistí ovládání různých zařízení, jako je osvětlení, motory, aktuátory či speciální efekty. Systém bude založen na využití platformy Raspberry Pi jako centrálního řídicího prvku a modulů ESP32 jako distribuovaných periferních jednotek. Komunikace bude realizována prostřednictvím protokolu MQTT a strukturovaného formátu JSON, který umožní snadno definovat jednotlivé scény a jejich parametry. Výsledkem práce bude funkční vzorek systému pro jednu místnost, který bude demonstrovat možnosti navrženého řešení, jeho modularitu a připravenost k rozšíření.

Cíle bakalářské práce:

1. Provedení rešerše problematiky, zejména protokolu MQTT, formátu JSON a využití platforem Raspberry Pi a ESP32 v oblasti IoT.
2. Návrh architektury systému se zajištěním modularity a možnosti budoucího rozšíření.
3. Návrh a implementace hardwarové a softwarové části systému pro jednu místnost s možností spouštění scén.
4. Experimentální ověření funkčnosti systému na demonstrační úloze.

Seznam doporučené literatury:

[1] VALÁŠEK, Michael. Mechatronika. Praha: České vysoké učení technické, 1995. ISBN 80-01-01276-X.

[2] HEROUT, Pavel. Učebnice jazyka C. 1. vydání. Brno: Computer Press, 2004. ISBN 80-251-0452-3

[3] MALÝ, Martin. ESP32 prakticky : od základních obvodů k pokročilým aplikacím. 1. vydání. Praha : CZ.NIC, z.s.p.o., 2024. 525 s. ISBN 978-80-88168-79-9.

Termín odevzdání bakalářské práce je stanoven časovým plánem akademického roku 2025/26

V Brně, dne

L. S.

prof. Ing. Jindřich Petruška, CSc. ředitel ústavu	doc. Ing. Jiří Hlinka, Ph.D. děkan fakulty

Abstrakt

Tato bakalárska práca sa zaoberá návrhom a realizáciou modulárneho riadiaceho systému pre interaktívne múzejné expozície s využitím platforiem Raspberry Pi a ESP32...

Summary

This bachelor thesis deals with the design and implementation of a modular control system for interactive museum exhibits using Raspberry Pi and ESP32 platforms...

Klíčová slova

IoT, Raspberry Pi, ESP32, MQTT, múzeum, riadiaci systém, mechatronika

Keywords

IoT, Raspberry Pi, ESP32, MQTT, museum, control system, mechatronics

Bibliografická citace

Citace tištěné práce:
ŽABKA, A. *Modulární systém pro automatizované řízení muzejních expozic.* Bakalářská práce. Ing. Michal Bastl, Ph.D. (vedoucí práce). Brno: Vysoké učení technické v Brně, Fakulta strojního inženýrství, 2026.

Citace elektronického zdroje:
ŽABKA, A. *Modulární systém pro automatizované řízení muzejních expozic.* Online, bakalářská práce. Ing. Michal Bastl, Ph.D. (vedoucí práce). Brno: Vysoké učení technické v Brně, Fakulta strojního inženýrství, 2026. Dostupné z: https://www.vut.cz/studenti/zav-prace/detail/174036. [cit. 2026-03-07].

Týmto čestne vyhlasujem, že som bakalársku prácu vypracoval samostatne...

Andrej Žabka

Brno . . . .

Na tomto mieste by som chcel poďakovať...

Andrej Žabka

Obsah

1 Úvod **9**

2 Súčasný stav problematiky **10**
2.1 Vývoj výstavníctva a interaktívne expozície . . . 10
2.2 Architektúry riadiacich systémov . . . 10
2.3 Porovnanie dostupných riadiacich platforiem . . . 11
2.3.1 Priemyselné riadiace systémy (PLC) . . . 11
2.3.2 Profesionálne show-control systémy . . . 11
2.3.3 Otvorené systémy na báze IoT platforiem . . . 12
2.4 Komunikačné protokoly . . . 12
2.4.1 Prenosové médiá: drôtové vs. bezdrôtové . . . 12
2.4.2 Aplikačné protokoly . . . 13
2.5 Hardvérové platformy pre IoT . . . 13
2.5.1 Jednodoskové počítače (SBC) . . . 13
2.5.2 Mikrokontroléry pre koncové uzly . . . 14
2.6 Výkonová elektronika pre fyzické efekty . . . 14
2.6.1 Riadenie jednosmerných (DC) motorov . . . 14
2.6.2 Spínanie sieťových a nízkonapäťových záťaží . . . 14
2.7 Prístupy k riadiacej logike expozície . . . 15
2.8 Dátové formáty pre konfiguráciu a prenos . . . 15
2.9 Zhrnutie rešerše . . . 16

3 Návrh architektúry systému **17**
3.1 Požiadavky na systém . . . 17
3.2 Systémová topológia a sieťová koncepcia . . . 17
3.3 Výber a rozdelenie hardvérových uzlov . . . 18
3.4 Softvérové jadro systému – konečný stavový automat . . . 19
3.5 Komunikačná vrstva – MQTT a formát scén . . . 19

4 Implementácia hardvérovej časti **21**
4.1 Centrálna jednotka – Raspberry Pi . . . 21
4.2 Distribuované uzly – ESP32 . . . 21
4.2.1 Motorický uzol – riadenie DC motorov . . . 21
4.2.2 Reléový uzol – spínanie záťaží . . . 22
4.2.3 Vstupný tlačidlový uzol . . . 23
4.3 Mechanická konštrukcia exponátov . . . 23

5 Implementácia softvérovej časti **25**
5.1 Firmvér ESP32 25
5.1.1 Spoločná infraštruktúra – sieť, watchdog a OTA 25
5.1.2 Tlačidlový uzol 26
5.1.3 Motorický uzol 26
5.1.4 Reléový uzol – spínanie záťaží 26
5.2 Backend Raspberry Pi 27
5.2.1 Orchestrátor systému – MuseumController 27
5.2.2 Stavový automat a engine scén 28
5.2.3 Mediálna vrstva – audio a video 29
5.2.4 Komunikačná vrstva MQTT 29
5.2.5 Webový dashboard 30
5.3 Konfigurácia scén – formát JSON 30

6 Experimentálne overenie **31**
6.1 Popis fyzického testovania a testovacieho prostredia 31
6.2 Testované scenáre a výsledky 31
6.2.1 Scenár 1 – Komplexná lineárna scéna s hardvérom 31
6.2.2 Scenár 2 – Dlhodobý test stability riadiaceho systému 32
6.2.3 Scenár 3 – Test sieťovej odolnosti a LWT (Last Will) 32
6.2.4 Scenár 4 – Meranie latencie MQTT spätnej väzby 32
6.3 Zhodnotenie overenia 33

7 Záver **34**

Zoznam obrázkov **36**

Zoznam tabuliek **37**

Použitá literatúra **38**

Použitá literatúra **38**

1 Úvod

Moderné výstavníctvo smeruje od pasívneho vystavovania k automatizácii expozičného priestoru. Namiesto statických popisok a izolovaných exponátov sa priestor stáva aktívnym prostredím, ktoré reaguje na prítomnosť návštevníka – koordinuje osvetlenie, zvuk, video a pohyb fyzických prvkov do jedného synchronizovaného celku. Takýto prístup je uplatniteľný univerzálne: rovnako pre zákaznicky navrhnuté interaktívne exponáty, ako aj pre bežné predmety obohatené o automatizované efekty. Riadiaci systém musí pritom podporovať nielen pevnú časovú postupnosť udalostí, ale podľa potreby aj dynamické vetvenie scenára na základe vstupov z prostredia.

Realizácia takýchto inštalácií však naráža na viacero technologických a finančných obmedzení. Komerčné show-control riešenia ponúkajú vysokú spoľahlivosť a funkcionalitu, avšak ich obstarávacie náklady sú pre menšie expozície často neúnosné [3]. Jednoduché časovače a lokálne ovládače naopak neposkytujú potrebnú úroveň interaktivity a flexibility. Problematická je aj kabeláž – v historických objektoch býva ťažké, prípadne nežiadúce viesť desiatky metrov vodičov k senzorom a aktuátorom [9].

Cieľom tejto bakalárskej práce je návrh a implementácia riadiaceho systému, ktorý uvedené problémy rieši kombináciou IoT technológií a vlastnej softvérovej architektúry. Výsledkom je systém postavený na stavovom automate, ktorý umožňuje definovať priebeh expozície presnejšie a flexibilnejšie.

Navrhovaná architektúra využíva bezdrôtový model klient–server, ktorý minimalizuje potrebu rozsiahlej kabeláže. Centrálnu riadiacu logiku zabezpečuje minipočítač Raspberry Pi. Ten komunikuje s distribuovanými uzlami na platforme ESP32 prostredníctvom protokolu MQTT. Správanie systému nie je pevne zakompilované vo firmvéri, ale je konfigurovateľné pomocou JSON súborov. Táto vlastnosť umožňuje jednoducho meniť scenár expozície bez zásahu do zdrojového kódu. Funkčnosť systému je v závere práce demonštrovaná na prototypovej miestnosti, ktorá integruje riadenie motorov, osvetlenia, dymostroja a multimediálneho obsahu.

2 Súčasný stav problematiky

Cieľom tejto kapitoly je analyzovať súčasný stav technických prostriedkov používaných v modernom výstavníctve, multimediálnych inštaláciách a automatizácii. Kapitola poskytuje prehľad dostupných hardvérových a softvérových platforiem s dôrazom na ich vhodnosť pre realizáciu interaktívnych expozícií. Zároveň sú diskutované kľúčové technické požiadavky na komunikačné protokoly v distribuovaných systémoch, prístupy k riadiacej logike expozície a vhodné dátové formáty pre konfiguráciu a prenos príkazov.

2.1 Vývoj výstavníctva a interaktívne expozície

Interaktívne prvky v múzeách nie sú novinkou – prvé mechanické systémy s preklápateľnými odpoveďami sa objavovali už v 70. rokoch 20. storočia [3]. S nástupom digitálnych technológií sa tieto prvky rozvinuli do systémov zahŕňajúcich dotykové obrazovky, virtuálnu a rozšírenú realitu, čím sa zásadne zmenili požiadavky na riadiacu infraštruktúru [4].

Rastúca technologická komplexnosť však so sebou priniesla aj nové prevádzkové výzvy. Ako upozorňuje súčasná literatúra, pokročilé interaktívne inštalácie si vyžadujú neustálu údržbu a aktualizáciu, čo naráža na finančné a personálne limity najmä v menších kultúrnych inštitúciách bez vyhradeného IT oddelenia [7]. Snaha o minimalizáciu prevádzkových výpadkov a zníženie závislosti od externých technikov tak prirodzene vytvorila tlak na zmenu návrhu riadiacich systémov. Do popredia sa dostáva požiadavka na modulárne architektúry, ktoré namiesto uzavretého jednoúčelového hardvéru umožňujú jednoduchšiu škálovateľnosť, vzdialenú diagnostiku a rekonfiguráciu bez nutnosti fyzického zásahu na mieste [8].

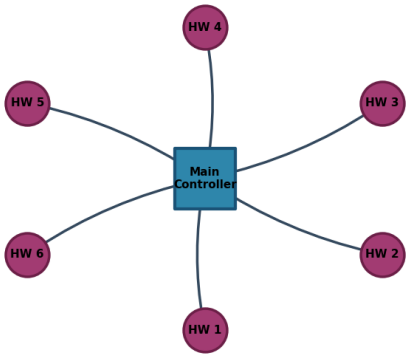
2.2 Architektúry riadiacich systémov

Voľba sieťovej a logickej architektúry riadiaceho systému priamo ovplyvňuje nielen náklady na inštaláciu, ale aj robustnosť a reakčnú dobu celého prostredia [6]. V praxi sa stretávame predovšetkým s dvoma základnými prístupmi:

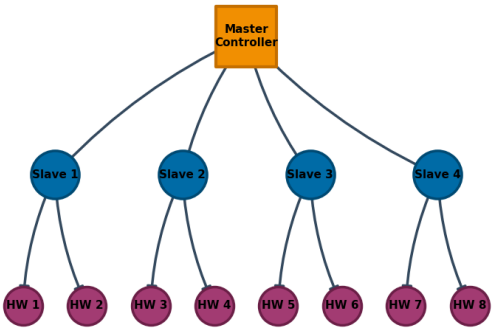
- **Centralizovaná topológia:** Jedna centrálna jednotka priamo ovláda všetky vstupné a výstupné periférie, zvyčajne cez zapojenie typu hviezda s individuálnymi vedeniami ku každému prvku. Výhodou je jednoduchá správa na jednom mieste, avšak vzniká takzvaný jediný bod zlyhania (*single point of failure*) a v historických budovách je táto topológia problematická kvôli rozsiahlej kabeláži [9].

- **Distribuovaná topológia (Master-Slave):** Inteligencia je rozložená do viacerých uzlov. Centrálna jednotka (master) komunikuje s periférmi (slaves), ktoré riešia svoju vlastnú časť logiky [10]. Každý periférny uzol disponuje vlastným mikrokontrolérom schopným spracovávať lokálne udalosti a komunikovať s centrálnou jednotkou len pri dôležitých stavových zmenách. Topológia tak umožňuje modulárne

rozširovanie systému a redukuje potrebu ťahať kabeláž z jedného riadiaceho bodu [6].



Obrázok 2.1: Centralizovaná topológia



Obrázok 2.2: Distribuovaná topológia

Výzvou pri návrhu riadiacich systémov pre expozície zostáva koordinácia viacerých asynchrónnych udalostí súčasne, čo si vyžaduje využitie formalizovaných metód, ako sú konečné stavové automaty (*Finite State Machine*, FSM) [6].

2.3 Porovnanie dostupných riadiacich platforiem

Samotný teoretický návrh topológie a riadiacej logiky je však len prvým krokom k funkčnej expozícii. Pre fyzickú implementáciu týchto konceptov do praxe je kľúčový výber vhodných hardvérových a softvérových prostriedkov. V súčasnosti neexistuje jedno univerzálne riešenie – realizátori si spravidla vyberajú medzi viacerými kategóriami, z ktorých každá má svoje špecifiká, výhody a obmedzenia. Výber konkrétnej platformy závisí od požiadaviek na determinizmus, schopnosť synchronizácie multimédií a v neposlednom rade od dostupného rozpočtu.

2.3.1 Priemyselné riadiace systémy (PLC)

V oblasti klasickej automatizácie sú dlhoročným štandardom Programovateľné logické automaty (PLC) od výrobcov ako **Siemens, Beckhoff či Mitsubishi** [1, 2]. Ich hlavnou výhodou je determinizmus, teda zaručená odozva systému na vstup v presne definovanom čase [1, 2]. Táto vlastnosť je kritická pre bezpečnostné funkcie, ale pre bežné interaktívne inštalácie v múzeách je často nadbytočná. PLC sú však primárne navrhnuté na prácu s logickými signálmi a narážajú na výrazné obmedzenia v oblasti multimédií. Absencia natívnych HDMI portov a neschopnosť priamo dekódovať video súbory vyžadujú použitie externých mediálnych prehrávačov [15]. Vysoká cena licencií a hardvéru navyše robí tieto systémy neprístupnými pre menšie kultúrne inštitúcie [3].

2.3.2 Profesionálne show-control systémy

Špecializované systémy ako **Medialon** a **Alcorn McBride** sú navrhnuté priamo pre potreby tematických parkov a múzeí [12, 13]. Na rozdiel od priemyselných PLC ich architektúra nie je postavená na cyklickom vykonávaní kódu, ale na princípe časovej osi (*timeline*), ktorá umožňuje operátorovi precízne definovať a synchronizovať udalosti v čase [14, 15]. Tieto systémy natívne integrujú protokoly zábavného priemyslu (SMPTE Time-

code, DMX512, OSC), ktoré sú optimalizované pre profesionálne produkčné prostredie [15]. Pre inštalácie v menších múzeách však táto špecializácia predstavuje skôr obmedzenie – podpora týchto protokolov vyžaduje certifikovaný hardvér a proprietárna povaha systému vytvára závislosť na dodávateľovi, čo znemožňuje flexibilné rozširovanie o vlastné hardvérové riešenia bez drahých certifikácií [16].

2.3.3 Otvorené systémy na báze IoT platforiem

Treťou, čoraz populárnejšou kategóriou sú otvorené a modulárne systémy postavené na princípoch Internetu vecí (IoT). Tento segment zaznamenal za posledné desaťročie prudký nárast vďaka dostupnosti jednodoskových počítačov (SBC) a výkonných mikrokontrolérov [17]. Na rozdiel od uzavretých systémov táto kategória využíva open-source softvér a štandardné programovacie jazyky (Python, C++), čo poskytuje vývojárom plnú kontrolu nad riadiacou logikou bez obmedzení uzavretých komerčných platforiem [5]. Architektúra je typicky rozdelená na dve funkčné úrovne [11]:

- **Výkonné centrálne jednotky (SBC):** Jednodoskové počítače s plnohodnotným operačným systémom (najčastejšie na báze Linuxu) slúžiace na koordináciu celého systému a predovšetkým na náročné multimediálne úlohy.

- **Distribuované periférie:** Koncové prvky (senzory, osvetlenie, mechanika) riadené samostatnými mikrokontrolérmi, ktoré fungujú ako inteligentné sieťové uzly schopné spracúvať dáta lokálne [6].

Využitie platforiem ako Raspberry Pi pre centrálnu logiku a ESP32 pre distribuované uzly umožňuje budovanie systémov, ktoré kombinujú výpočtovú silu potrebnú pre multimédiá s riadením hardvéru [11]. Tento prístup je nielen finančne efektívny, ale vďaka rozsiahlej komunite a dostupnosti knižníc umožňuje rýchle prototypovanie a iteratívny vývoj riadiacej logiky [11, 3].

2.4 Komunikačné protokoly

V prípade využitia otvorenej IoT architektúry, ktorá fyzicky oddeľuje centrálnu riadiacu jednotku od distribuovaných periférnych uzlov, vzniká kritická potreba ich vzájomnej koordinácie. V takomto distribuovanom systéme je voľba komunikačného protokolu rozhodujúcim faktorom pre celkovú plynulosť interakcie [11]. Pri výbere vhodného riešenia sa zvažujú parametre ako latencia, réžia protokolu (*overhead*) a spoľahlivosť doručenia správ v rámci lokálnej siete [18].

2.4.1 Prenosové médiá: drôtové vs. bezdrôtové

Pri inštaláciách v múzeách je kritickým faktorom spôsob prepojenia komponentov. Tradičné drôtové zbernice (RS-485, Ethernet) ponúkajú vysokú spoľahlivosť a deterministickú komunikáciu. Ich nevýhodou je nutnosť fyzickej kabeláže ku každému prvku, čo môže byť v historických priestoroch problematické [9]. Bezdrôtové technológie (Wi-Fi, Bluetooth, Zigbee) eliminujú potrebu kabeláže a výrazne zvyšujú flexibilitu systému [11]. Mikrokontroléry ESP32 majú vstavané Wi-Fi a priamo sa pripájajú do lokálnej siete, čo umožňuje jednoducho premiestňovať interaktívne prvky [26].

2.4.2 Aplikačné protokoly

Tradičný protokol **HTTP** sa pre interaktívne IoT systémy javí ako nevhodný: model request/response vyžaduje neustále dopytovanie servera, čo vedie k vysokej latencii a zbytočnej réžii pri prenose krátkych riadiacich príkazov [19, 20]. **WebSocket** síce umožňuje trvalé obojsmerné spojenie, jeho implementácia na jednoduchých mikrokontroléroch je však pamäťovo náročná [21]. **MQTT** (*Message Queuing Telemetry Transport*), definovaný ako ISO štandard ISO/IEC 20922:2016, je v súčasnosti najrozšírenejším protokolom pre distribuované IoT systémy [22]. Funguje na modeli *Publish/Subscribe*, kde centrálny sprostredkovateľ (broker) distribuuje správy prihláseným klientom na základe hierarchických tém (topics) [23]. Prenos správ prebieha nad trvalým TCP/IP spojením.

Experimentálne štúdie potvrdzujú, že MQTT v lokálnom sieťovom prostredí výrazne prekonáva alternatívne protokoly z hľadiska latencie a stability [21]. Priemerná odozva 11,04 ms pri použití MQTT QoS 0 je 3,76-násobne nižšia v porovnaní s protokolom WebSocket (41,54 ms), čo je kľúčové pre synchrónne efekty, kde by akékoľvek zreteľné oneskorenie narušilo imerziu návštevníka [21]. Protokol MQTT definuje tri úrovne kvality služieb (QoS), ktoré umožňujú prispôsobiť komunikáciu podľa požiadaviek [23]:

Tabuľka 2.1: Úrovne kvality služby (QoS) v MQTT

QoS	Úroveň	Použitie
0	At most once	Menej kritické dáta (napr. pravidelné senzorické hodnoty), bez potvrdenia
1	At least once	Dôležité správy, možné duplikáty, s potvrdením
2	Exactly once	Kritické povely (napr. pohyb motora), garantované doručenie presne raz

Okrem QoS poskytuje MQTT mechanizmus *Last Will and Testament* (LWT), ktorý umožňuje brokeru automaticky informovať ostatné časti systému, ak dôjde k neočakávanému odpojeniu zariadenia [24]. Tento mechanizmus je v kombinácii so zostatkovými (*retained*) správami kľúčový pre budovanie samoopravných systémov, ktoré dokážu po výpadku napájania okamžite obnoviť svoj stav [24, 23].

2.5 Hardvérové platformy pre IoT

Samotný návrh distribuovanej architektúry a komunikácie si v praxi vyžaduje nasadenie adekvátnych hardvérových komponentov. Realizácia takéhoto riadiaceho systému sa typicky delí na dve špecifické úrovne: centrálnu jednotku určenú na orchestráciu systému a multimediálne funkcie, a distribuované koncové uzly obsluhujúce senzory a aktuátory priamo v priestore expozície.

2.5.1 Jednodoskové počítače (SBC)

Jednodoskové počítače (SBC), reprezentované platformou **Raspberry Pi**, predstavujú vhodné riešenie pre rolu centrálneho koordinátora expozície, keďže umožňujú beh plnohodnotného operačného systému a správu komplexnej riadiacej logiky [11]. Pre potreby moderných expozícií je rozhodujúca natívna podpora multimediálnych rozhraní a priameho hardvérového výstupu obrazu a zvuku [25]. Práve táto vlastnosť predstavuje zá-

sadnú výhodu oproti priemyselným PLC, ktoré si pre rovnaké funkcie vyžadujú externé mediálne prehrávače [15]. Dostupnosť GPIO rozhrania navyše umožňuje pripojiť rôzne vstupy a výstupy. [25].

2.5.2 Mikrokontroléry pre koncové uzly

Historicky populárna platforma **Arduino Uno** (8-bitový čip ATmega328P) sa v kontexte moderného IoT stáva limitujúcou [27, 26]. S taktom 16 MHz a 2 kB RAM nie je schopná efektívne spracovávať sieťový TCP/IP stack a súčasne parsovať komplexné JSON správy, čo vedie k nestabilite systému pri vyššej sieťovej záťaži [27, 26]. Nástup čipu **ESP32** od spoločnosti Espressif priniesol veľkú zmenu v oblasti IoT periférií. Integrácia Wi-Fi konektivity priamo v čipe eliminuje potrebu externých sieťových modulov, čo zjednodušuje hardvérový návrh uzlov [26]. Dvojjadrová architektúra je pre interaktívne expozície výhodou: jedno jadro môže byť vyhradené výhradne pre sieťovú komunikáciu (správa MQTT spojenia, Wi-Fi stack), zatiaľ čo druhé jadro v reálnom čase obsluhuje kritické hardvérové úlohy, ako je PWM riadenie motorov alebo čítanie senzorov. Táto separácia eliminuje nežiaduci jitter pri spracovaní časovo kritických úloh, ktorý by inak vznikal pri súbežnej obsluhe sieťovej réžie [26]. Na rozšírenie výstupnej kapacity sa využívajú I2C expandéry (napr. PCF8574), ktoré cez dvojvodičovú zbernicu sprístupňujú dodatočné výstupy bez obsadenia všetkých GPIO pinov.

2.6 Výkonová elektronika pre fyzické efekty

Riadiaci systém musí byť schopný bezpečne a efektívne ovládať výkonové prvky rôznych napäťových hladín – od jednosmerných motorov a nízkonapäťových záťaží (typicky 12 V DC) až po zariadenia pracujúce na sieťovom napätí 230 V AC.

2.6.1 Riadenie jednosmerných (DC) motorov

Na obojsmerné riadenie DC motorov sa využívajú H-mostíky. Staršie obvody na báze Darlingtonových tranzistorov (napr. L293D alebo L298N) sú v súčasnosti pre moderné aplikácie nevhodné pre ich vysoký úbytok napätia (typicky nad 1,5 V), čo spôsobuje tepelné straty a vyžaduje prídavné chladenie [28]. Súčasným štandardom sú preto MOSFET H-mostíky, ktoré vďaka minimálnemu odporu v zopnutom stave takmer úplne eliminujú nežiaduce zahrievanie. Pre nízkoenergetické aplikácie sa využívajú kompaktné čipy s vstavanými ochranami (napr. DRV88xx), zatiaľ čo pre zvládanie rozbehových špičiek v rádoch desiatok ampérov slúžia vysokoprúdové moduly typu BTS7960 [29].

Pre jednosmerné spínanie a PWM reguláciu záťaží, ktoré presahujú prúdový limit GPIO pinov (jednotky miliampérov), sa využíva samostatný MOSFET modul. Pri takomto zapojení riadi GPIO pin výhradne hradlo (*gate*) tranzistora. Celý výkonový prúd tak tečie oddelene z externého zdroja, čím je mikrokontrolér bezpečne chránený pred preťažením [28].

2.6.2 Spínanie sieťových a nízkonapäťových záťaží

Pre ovládanie svetelných efektov, dymostrojov alebo iných záťaží v expozíciách je nevyhnutné oddeliť nízkonapäťovú riadiacu logiku od výkonových obvodov. Reálne inštalácie pritom kombinujú záťaže na rôznych napäťových hladinách – bežne 230 V AC (sieťové spotrebiče) aj 12 V DC (LED pásy, malé aktuátory). Pri výbere spínacej technológie sa

v praxi porovnávajú dve základné kategórie: **polovodičové relé** (SSR – *Solid State Relay*) a **elektromechanické relé** [30].

SSR nevyužívajú pohyblivé kontakty, čím eliminujú mechanické opotrebovanie a akustický hluk. Ich nevýhodou je však obmedzená univerzálnosť – rôzne typy SSR (napr. *zero-cross* alebo *random turn-on*) sú optimalizované pre špecifické typy záťaží a nie všetky typy podporujú súčasne AC aj DC obvody [31, 32]. Elektromechanické relé naopak poskytujú galvanickú izoláciu a sú schopné spínať záťaže rôzneho charakteru bez obmedzenia, za cenu pohyblivých kontaktov s obmedzenou životnosťou pri vyššej frekvencii spínania [30].

2.7 Prístupy k riadiacej logike expozície

Softvérová vrstva riadiaceho systému určuje, akým spôsobom sa definuje a vykonáva scenár expozície. V praxi sa využívajú predovšetkým dva základné prístupy, ktoré sa líšia schopnosťou reagovať na externé udalosti. Najjednoduchším riešením je **lineárny (sekvenčný) prístup**: scenár je pevná séria akcií vykonávaných v pevnom poradí a čase, bez možnosti vetvenia. Takéto systémy sú jednoduché na implementáciu a postačujú pre pasívne inštalácie, kde sa obsah prehráva v slučke bez závislosti od vstupu návštevníka [5]. Ich zásadným obmedzením je neschopnosť reagovať na externé podnety počas čakania na ďalší krok scenára, čo pri komplexnejšom scenári vedie k blokovaniu celého systému [5].

Pre interaktívne expozície, kde scenár musí reagovať na vstupy návštevníka (napr. stlačenie tlačidla, dohratý mediálny klip), je vhodnejší prístup založený na **konečnom stavovom automate** (*Finite State Machine*, FSM). V tomto modeli je správanie systému rozdelené do diskrétnych stavov; prechody medzi nimi sú podmienené konkrétnymi udalosťami (*events*) – uplynutím časovača, prijatím MQTT správy alebo dokončením prehrávania média [6]. FSM umožňuje neblokujúce čakanie na viacero rôznych podmienok súčasne, čo je nevyhnutné pre dynamické vetvenie scenára [5].

2.8 Dátové formáty pre konfiguráciu a prenos

Pre štruktúrovanie prenášaných dát a ukladanie konfigurácie je nutné zvoliť vhodný formát. V kontexte IoT a webových technológií sa najčastejšie porovnávajú textové formáty XML, YAML a JSON [33].

XML(*eXtensible Markup Language*) je dlhoročným priemyselným štandardom pre výmenu dát. V prostredí embedded systémov je však považované za zastarané kvôli extrémnej verbosite a zložitosti parserov, ktoré neúmerne zaťažujú pamäť mikrokontroléra [33].

YAML (*YAML Ain't Markup Language*) si získal obľubu v oblasti DevOps vďaka podpore komentárov a maximálnej čitateľnosti pre človeka [34]. Avšak YAML parsery pre C++ sú pamäťovo náročné a často chybové pri spracovávaní neštandardných dát, čo z neho robí rizikovú voľbu pre real-time komunikáciu na mikrokontroléroch [35].

JSON(*JavaScript Object Notation*) sa stal de facto štandardom pre IoT vďaka optimálnej rovnováhe medzi čitateľnosťou a strojospracovateľnosťou [33]. Pre mikrokontroléry s obmedzenou RAM existujú vysoko optimalizované parsery vo forme knižníc, ktoré minimalizujú fragmentáciu pamäte pri spracovaní JSON správ [36]. Okrem prenosu dát v IoT komunikácii nachádza JSON uplatnenie aj ako **konfiguračný jazyk pre definíciu riadiacej logiky**. Vďaka svojej štruktúre umožňuje hierarchicky popisovať stavy, podmienky

prechodov a časové postupnosti akcií v podobe textového súboru editovateľného bez rekompilácie softvéru [33]. Tento prístup oddeľuje konfiguráciu od kódu aplikácie, čo znižuje závislosť na vývojároch pri bežných zmenách logiky systému [37].

2.9 Zhrnutie rešerše

Analýza dostupných technických riešení preukázala, že pre špecifické potreby modulárnej a cenovo dostupnej muzeálnej inštalácie nie sú optimálne ani priemyselné PLC (vysoká cena, absencia multimediálnych rozhraní), ani uzavreté komerčné show-control systémy (závislosť na dodávateľovi, obmedzená rozšíriteľnosť) [1, 2, 16]. Najvhodnejším prístupom je distribuovaný IoT systém postavený na otvorených platformách, ktorý poskytuje flexibilitu pre implementáciu nelineárnych naratívov pri zachovaní nízkych nákladov [5, 3].

Z hľadiska jednotlivých technických vrstiev rešerš identifikovala nasledujúce závery. Protokol MQTT sa ukázal ako jednoznačne vhodný pre distribuované systémy s požiadavkou na nízku latenciu [21]. Centrálna SBC jednotka pritom nesie zodpovednosť nielen za MQTT orchestráciu, ale aj za synchrónne prehrávanie audiovizuálneho obsahu [11]. Pre riadiacu logiku interaktívnych expozícií je vhodný model konečného stavového automatu (FSM), ktorý umožňuje neblokujúce vetvenie scenára na základe externých udalostí [6]. Konfigurácia vo formáte JSON poskytuje ľahko editovateľné definície stavov a prechodov bez nutnosti zásahu do zdrojového kódu.

3 Návrh architektúry systému

Táto kapitola popisuje celkovú architektúru navrhovaného riadiaceho systému. Vychádza z analýzy dostupných technológií uvedenej v predchádzajúcej kapitole a z požiadaviek definovaných v sekcii 3.1.

3.1 Požiadavky na systém

Požiadavky vychádzajú priamo z cieľov definovaných v zadaní práce a z potrieb konkrétnej inštalácie, pre ktorú bol systém navrhnutý a implementovaný.

Z hľadiska **funkčných požiadaviek** sa očakáva najmä bezproblémová integrácia a synchronizácia rôznorodých hardvérových prvkov:

- **Riadenie motora a výkonu:** Primárnou úlohou je riadenie výkonových prvkov, špecificky jednosmerných motorov s riadením ich rýchlosti a smeru otáčania. Ďalšou kľúčovou funkciou je priame spínanie záťaží pracujúcich na sieťovom napätí **230 V** (osvetlenie, dymostroje) a taktiež **12 V**.
- **Multimediálna synchronizácia:** Keďže moderné expozície stavajú aj na audiovizuálnych vnemoch, systém musí zabezpečiť synchronizované prehrávanie zvuku a videa.
- **Interaktivita a časová os (*timeline*):** Systém musí vedieť na základe hardvérových vstupov zmeniť vetvenie prebiehajúcej scény, no zároveň podporovať chronologické časovanie sekvencií.

Medzi **nefunkčné požiadavky** s ohľadom na udržateľnosť patria:

- **Modularita a softvérové oddelenie:** Pre zachovanie oddelenia softvérového inžinierstva od obsahovej tvorby sa úprava logiky (scenára) musí dať vykonať výhradne modifikáciou **textových konfiguračných súborov**. Tým sa odstraňuje nutnosť zásahu do zdrojového kódu v pythone.
- **Dostupnosť:** Vzhľadom na finančné limity inštalácií musí byť systém postavený na komerčne dostupných, cenovo výhodných mikrokontroléroch a **Internet of Things (IoT)** platformách namiesto drahých priemyselných automatov (PLC).

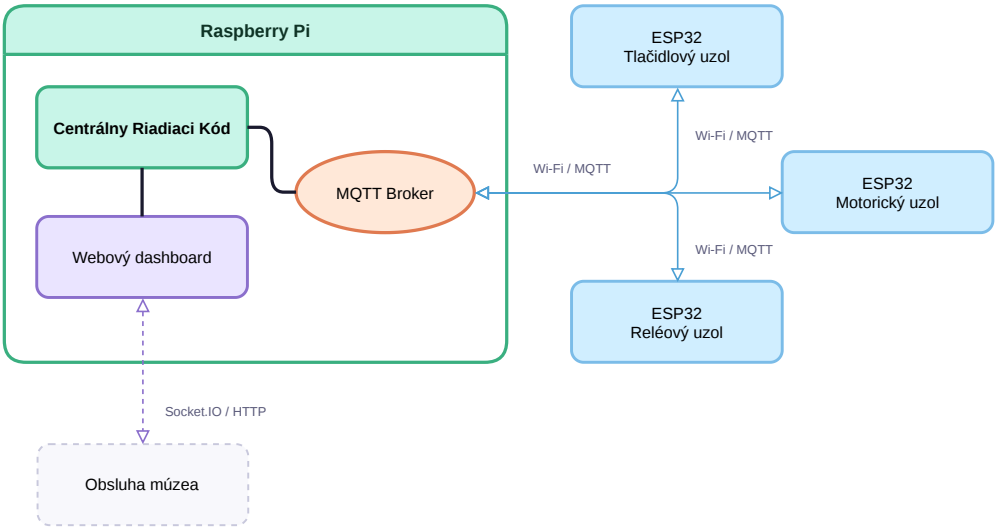
3.2 Systémová topológia a sieťová koncepcia

Na architektúru navrhovaného systému je nevyhnutné nazerať z dvoch odlišných, no vzájomne sa dopĺňajúcich hľadísk: z **logického (výpočtového)** a z **fyzického (sieťového)**.

Z logického hľadiska ide o **distribuovanú architektúru** (viď kapitola 2.2). Riadiaca logika nie je sústredená výhradne v jednom centrálnom procesore, ale je rozdelená medzi viacero častí systému a presunutá bližšie k výkonovým prvkom. Koncové periférie postavené na mikrokontroléroch preto neslúžia len ako pasívne spínače, ale ako aktívne prvky,

ktoré lokálne spracúvajú prijaté príkazy a samostatne vykonávajú im pridelené fyzické akcie.

Z fyzického a sieťového hľadiska však systém dodržiava **hviezdicovú topológiu (*Star topology*)**. Hoci sú distribuované uzly plne inteligentné, nekomunikujú vzájomne medzi sebou (netvoria *Mesh* sieť). Akýkoľvek dátový tok na sieti vedie striktne obojsmerne len cez centrálny uzol.



Obrázok 3.1: Fyzická topológia riadiaceho systému

3.3 Výber a rozdelenie hardvérových uzlov

Systém je rozdelený do dvoch samostatných kategórií: na jeden centrálny uzol a na viacero výkonných periférií. Z hľadiska spoľahlivosti a rýchlosti prenosu medzi nimi by bolo ideálne zabezpečiť komunikáciu výhradne plne drôtovým spojením. V praxi však v expozíciách či *escape room* inštaláciách často nie je technologicky alebo esteticky možné naťahovať dodatočné káblové zväzky naprieč miestnosťou. Z tohto dôvodu bolo na distribúciu dátových príkazov z centrály zvolené bezdrôtové Wi-Fi pripojenie.

Dvojúrovňová architektúra bola fyzicky obsadená nasledovným hardvérom:

- **Centrálny uzol („Mozog“):** Hlavnú riadiacu jednotku systému tvorí jednodoskový počítač **Raspberry Pi**. Ako bolo analyzované v kapitole 2.5.1, táto platforma poskytuje dostatočný výpočtový výkon pre súčasnú prevádzku riadiacej logiky aj prehrávanie multimediálneho obsahu, pričom natívne hardvérové rozhrania umožňujú priame prepojenie s displejom a reproduktormi bez externých adaptérov.

- **Distribuované uzly („Svaly a zmysly“):** Na plnenie fyzických úkonov slúži modulárna sieť bezdrôtových mikrokontrolérov **ESP32**, ktoré sú implementované a napájané priamo pri interaktívnych objektoch. Tieto uzly sa na základe svojho zamerania softvérovo rozdeľujú na:

  – **Senzorický uzol** pre hardvérovú detekciu vstupov od používateľov.

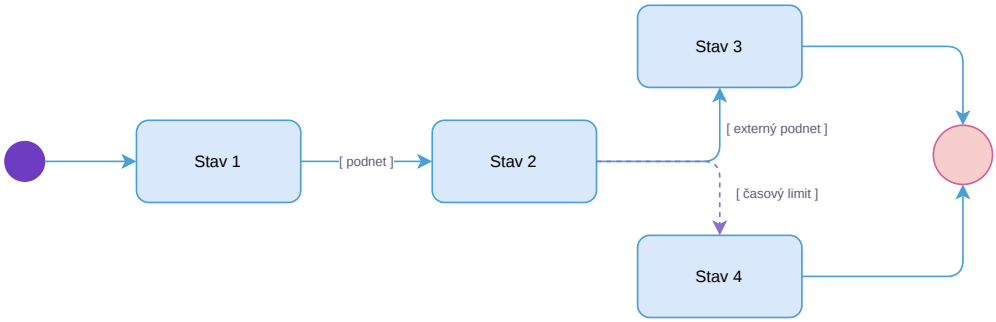
  – **Motorický uzol** pre ovládanie fyzického pohybu exponátov (jednosmerné motory).

– **Reléový uzol** pre spínanie efektov a osvetlenia či iných zariadení pracujúcich na 12 V a sieťovom napätí 230 V.

3.4 Softvérové jadro systému – konečný stavový automat

Ústrednou požiadavkou na softvérové jadro bola schopnosť **vetviť priebeh expozície** na základe reálnych podnetov od návštevníka. Scenár preto nesmel byť pevnou lineárnou sekvenciou akcií – systém musel v ľubovoľnom momente vedieť čakať na vonkajší vstup (napr. stlačenie tlačidla) a podľa jeho výsledku zvoliť ďalší smer deja. Klasický sekvenčný prístup je na takúto úlohu nevhodný z dôvodov analyzovaných v kapitole 2.7.

Z tohto dôvodu bolo softvérové jadro navrhnuté ako **Konečný stavový automat (FSM – *Finite State Machine*)**. Priebeh expozície je rozdelený do samostatných **stavov**, pričom systém nepretržite sleduje podmienky prechodu – vypršanie časovača, dokončenie prehrávania média, alebo príchod správy od hardvérovej periférie – a podľa nich plynulo presúva chod scenára do ďalšieho stavu. Podrobná softvérová implementácia tejto architektúry je rozpracovaná v Kapitole 5.



Obrázok 3.2: Diagram Konečného stavového automatu

3.5 Komunikačná vrstva – MQTT a formát scén

Pre výmenu dát medzi centrálnym Raspberry Pi a distribuovanými periférami (ESP32) bol zvolený **protokol MQTT** (*Message Queuing Telemetry Transport*) (viď kapitola 2.4.2). Prenos správ prebieha na koncepte *Publish/Subscribe* cez trvalé TCP/IP spojenie s centrálnym sieťovým sprostredkovateľom (*brokerom*).

Navrhnutý model siete implementuje niekoľko kľúčových komunikačných štandardov:

- **Hierarchické adresovanie zariadení:** Každé zariadenie je adresované vlastným topikom v hierarchickom formáte (napr. `room1/motor1`). Raspberry Pi zverejní príkaz na daný topik; ESP32 mikrokontrolér, ktorý má tento topik odoberaný, správu prijme a vykoná požadovanú akciu.

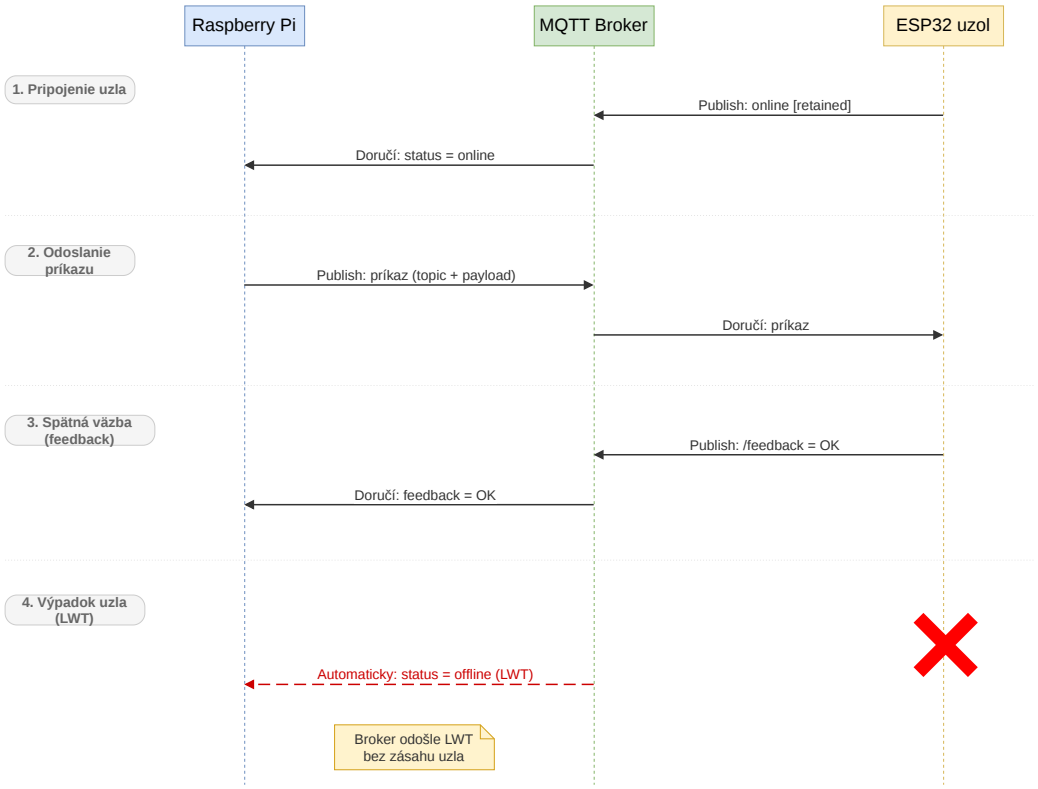
- **Spätná väzba (*feedback*):** Po vykonaní príkazu periférny uzol odošle potvrdzovaciu správu späť na topik s príponou `/feedback` (napr. `room1/motor1/feedback`). Centrála tak vie, že príkaz bol skutočne doručený a fyzicky vykonaný.

- **Detekcia dostupnosti (status + LWT):** Každý ESP32 uzol pri pripojení zverejní svoj stav na dedikovanom stavovom topiku ako zostatkovú (*retained*) správu – centrála tak vždy pozná aktuálny stav siete. Súčasne je nastavená správa *Last Will*

and Testament (LWT), ktorú broker automaticky zverejní v prípade neočakávaného výpadku spojenia.

- **Konfigurácia scény ako externý JSON súbor** (viď kapitola 2.8)**:** Celá logika scenára - čo má každý exponát vykonať, v akom poradí a za akých podmienok – je definovaná výhradne v externom súbore `.json`, nie priamo v kóde enginu. Obsluha môže scenár upravovať alebo vymeniť za behu programu bez akéhokoľvek zásahu do zdrojového kódu.

Toto oddelenie konfigurácie od kódu enginu umožňuje upravovať logiku a priebeh expozície bez zásahu do zdrojového kódu systému.



Obrázok 3.3: Sekvenčný diagram toku MQTT správ

4 Implementácia hardvérovej časti

Táto kapitola popisuje hardvérovú realizáciu navrhnutého riadiaceho systému. Jednotlivé uzly sú rozdelené podľa svojej funkčnej zodpovednosti — centrálna jednotka zabezpečuje riadiacu logiku a multimediálny výstup, distribuované uzly na báze ESP32 zabezpečujú ovládanie motorov, relé a senzorickú detekciu vstupu návštevníka.

4.1 Centrálna jednotka – Raspberry Pi

Úlohu hlavného komunikačného a multimediálneho uzla plní Raspberry Pi (viď kapitola 2.5.1). Z hľadiska fyzickej inštalácie bol umiestnený v tesnej blízkosti audiovizuálnej techniky (priamo za LCD displejom). Toto riešenie minimalizuje dĺžku káblových zväzkov, keďže Raspberry Pi disponuje HDMI portom ale aj audio výstup cez 3,5 mm jack. Pre integráciu do systému postačuje len bežný Ethernet kábel pre prístup do LAN siete. SEM EDITNUT TEXT, PRIDAM OBRAZOK KTORY UKAZE AKO JE ZAPOJENE RPI.... SCHEMA LEN + MOZNO AKO TO MAM IRL Na napájanie Raspberry Pi je využívaný USB-C adaptér - 5,0 V / 3,0 A.

[MIESTO PRE OBRÁZOK: Fotografia zapojeného Raspberry Pi s HDMI a LAN káblom, umiestneného za LCD displejom]

4.2 Distribuované uzly – ESP32

Riadiaca logika expozície bola decentralizovaná do niekoľkých samostatných uzlov postavených na architektúre mikrokontrolérov rodiny ESP32 (viď kapitola 2.5.2). Tento prístup fyzicky rozdeľuje zodpovednosť do funkčných blokov a znižuje potrebu viesť káblové zväzky naprieč miestnosťou. Všetky uzly komunikujú s Raspberry Pi cez Wi-Fi sieť.

Riadiace a silové uzly (pre ovládanie motorov a spínanie relé) boli umiestnené spoločne vo vnútri inštalačného rozvádzača integrovaného priamo do fyzického exponátu expozície. Senzorický tlačidlový uzol pre používateľský vstup bol namontovaný oddelene pri vstupe do miestnosti.

4.2.1 Motorický uzol – riadenie DC motorov

Pre ovládanie pohonov exponátov bola zvolená dvojica komponentov ESP32 DevKit V1 a výkonový budič BTS7960, ktorých kombinácia pokrýva celý rozsah požiadaviek od riadiacej logiky až po silové prúdové špičky pri rozbehoch. Konkrétne fyzické exponáty, ktoré tento uzol poháňa, sú popísané v kapitole 4.3.

Hardvérová implementácia uzla pozostáva z nasledujúcich komponentov:

- **Mikrokontrolér ESP32 DevKit V1:** Funguje ako mozog uzla, ktorý spracováva príkazy z MQTT brokera a tým generuje PWM signál pre ovládanie motorov.

- **2x Výkonový budič BTS7960:** Polovodičový H-mostík slúži na riadenie výkonu motorov (viď kapitola 2.6.1). Tento masívne dimenzovaný obvod bezpečne odoláva vysokým prúdovým špičkám pri rozbehoch (až do 43 A).

[MIESTO PRE OBRÁZOK: Schéma zapojenia uzla pre DC Motory (ESP32, BTS7960, 12V zdroj)]

4.2.2 Reléový uzol – spínanie záťaží

Na spínanie záťaží ostatných prvkov expozície slúži vyhradený reléový uzol:

- **Priemyselný I/O modul Waveshare:** Základom tohto uzla je komerčný hotový modul s čipom **ESP32-S3**. Obsahuje integrovanú podporu pre *Power over Ethernet* (PoE) a zbernicu RS485.

- **10 A mechanické relé:** Modul disponuje ôsmimi nezávislými relé. Spínajú priamo striedavých 230 V na chod dekoratívnej **Edisonovej žiarovky**, **výrobníka dymu** (ktorý vháňa paru do rozvedeného potrubia) a tiež prúd pre **svetlo simulujúce oheň** v kotle. Ďalšou vyhradenou sekciou je spínanie jednosmernej 12 V vetvy určenej na napájanie podsvietenia dvoch oddelených skupín **industriálnych analógových budíkov** a skrytých dizajnových LED pásov.

- **Dôvod nasadenia modulu:** Namiesto lokálne osadených polovodičových relé (SSR) (viď kapitola 2.6.2) bola na zaistenie požiarnej a elektrickej bezpečnosti použitá táto certifikovaná hotová doska. V prípade hardvérového zlyhania v múzejnej prevádzke umožňuje servisným technikom okamžitú, jednoduchú a bezpečnú výmenu „kus za kus“.

[MIESTO PRE OBRÁZOK: Schéma zapojenia relé uzla (Waveshare ESP32-S3, 230V obvody a 12V obvody)]

4.2.3 Vstupný tlačidlový uzol

Fyzický vstup zameraný na interakciu s návštevníkmi je spracovaný samostatným uzlom:

- **Samostatný mikrokontrolér:** Na vstupnom mieste pred stenou je fyzicky osadená inštalačná krabička. Aby si zachovala kompaktné vonkajšie zapojenie, disponuje vlastným DC napájacím konektorom pre privedenie 5 V napätia k vloženej doske **ESP32 DevKit V1**.
- **Antivandal tlačidlo:** Kvôli zabezpečeniu odolnosti pred opakovaným a tvrdým návštevníckym stláčaním bol zvolený priemyselný **19 mm vodotesný kovový spínač**, ktorý je prepojený s krabičkou riadiaceho modulu pomocou leteckého konektora typu **GX12 (4-pin)**. Cez tento štvoritý konektor sú súčasne vedené všetky vodiče uzla: signálový vodič tlačidla, napájanie LED prstenca, uzemnenie a referenčné 5 V. Spínač slúži ako prvotný štartovací impulz pre chod interaktívnej scény.
- **Riadenie LED prstenca cez MOSFET spínač:** Záťaž LED prstenca bola namiesto priameho napájania z GPIO pinu riadená cez kompaktný MOSFET PWM spínací modul s tranzistorom F5305S (viď kapitola 2.6.1), čím bola výstupná periféria mikrokontroléra ochránená pred prúdovým preťažením a zároveň bolo umožnené PWM stmievanie prstenca.

[MIESTO PRE OBRÁZOK: Fotografia inštalačnej krabičky tlačidlového uzla s viditeľným MOSFET modulom, ESP32 doskou a GX12 konektorom]

[MIESTO PRE OBRÁZOK: Schéma zapojenia vstupného tlačidlového uzla – ESP32, externý pull-up rezistor na GPIO 32, MOSFET F5305S pre napájanie LED prstenca a GX12 konektor]

4.3 Mechanická konštrukcia exponátov

Fyzické exponáty inštalácie CHRONOS boli navrhnuté v prostredí **Fusion 360**. Celková sústava obsahuje **24 ozubených kolies** z preglejky s jednotným modulom $m = 8$. Pre každé koleso bol vytvorený samostatný 3D model; pri menších kolesách bol G-kód generovaný a obrobky boli vyfrézované na dostupnej CNC fréze. Väčšie kolesá boli zadané externej firme, ktorá disponuje strojom s väčším pracovným priestorom.

Súčasťou návrhu bol aj **dekoratívny piest** pripevnený na sústave kolies, ktorý vykonáva pohyb bez mechanického zaťaženia, a **hodinový mechanizmus** s priemerom 1,3 m vrátane planetového prevodu s vypočítaným počtom zubov pre požadovaný prevodový

pomer. Montáž a povrchovú úpravu exponátov zabezpečila externá firma. Podrobná konštrukčná dokumentácia (výkresy, modely, prevodové výpočty) presahuje rámec tejto práce a nie je jej predmetom.

5 Implementácia softvérovej časti

Softvérová vrstva systému bola rozdelená do dvoch samostatných celkov, ktoré spolu komunikujú výhradne prostredníctvom protokolu MQTT. Prvý celok tvorí firmvér mikrokontrolérov ESP32, ktorý zabezpečuje priame riadenie hardvéru popísaného v kapitole 4. Druhý celok predstavuje backend Raspberry Pi, ktorý implementuje logiku scén, spravuje mediálny obsah a poskytuje webové rozhranie pre obsluhu. Celková architektúra oboch celkov a ich vzájomná komunikačná vrstva boli navrhnuté v kapitole 3; táto kapitola sa venuje konkrétnej realizácii jednotlivých komponentov.

5.1 Firmvér ESP32

Všetky tri uzly systému (tlačidlový, motorický a reléový) zdieľajú spoločnú infraštruktúrnu vrstvu, ktorá rieši sieťovú konektivitu, správu spojenia a bezpečnostnú odolnosť zariadenia. Táto vrstva bola zámerme oddelená od aplikačnej logiky uzla, čo umožňuje jednoduché rozšírenie systému bez nutnosti opakovane implementovať tie isté mechanizmy. Fyzické zapojenie a výber konkrétnych pinov jednotlivých uzlov sú podrobne popísané v kapitole 4.2.

5.1.1 Spoločná infraštruktúra – sieť, watchdog a OTA

Správa WiFi a MQTT pripojenia. Pri štarte sa uzol pripojí k sieti pomocou nakonfigurovaného SSID a hesla. Pri strate spojenia sa automaticky spustí mechanizmus opätovného pripájania s exponenciálnym nárastom čakacej doby od 3 s do maximálne 30 s, aby uzol nezahlcoval sieť zbytočnými pokusmi. Po obnovení WiFi sa MQTT klient znova inicializuje a uzol sa prihlási na odber svojich topicov.

Každý uzol pri prihlásení na MQTT broker publikuje správu `online` na status topic `devices/<clientID>/status` s príznakom `retain = true`. Zároveň bola nastavená správa Last Will Testament (LWT), ktorú broker automaticky odošle (`offline`) pri neočakávanom výpadku spojenia. Tento mechanizmus umožňuje backendu sledovať stav každého uzla v reálnom čase, ako je podrobnejšie popísané v kapitole 3.5.

Watchdog Timer. Aby sa predišlo situácii, kedy uzol zamrzne a prestane reagovať na príkazy, bol aktivovaný hardvérový Watchdog Timer (WDT). Časový limit je pre každý typ uzla nastavený individuálne — pre tlačidlový a reléový uzol na 30 sekúnd, pre motorický uzol na 60 sekúnd z dôvodu dlhších rampových operácií. Hlavná slučka programu pravidelne WDT resetuje. Pokiaľ k resetu nedôjde, napríklad pri zaseknutí v blokujúcej operácii, WDT vyvolá reštart zariadenia.

OTA aktualizácie. Všetky uzly boli vybavené podporou aktualizácie firmvéru cez sieť (Over-the-Air, OTA) bez potreby fyzického prístupu k zariadeniu. Pri spustení OTA procesu sa automaticky zastavia všetky aktívne výstupy a deaktivuje sa WDT, aby proces nahrávania nebol prerušený. Po úspešnej aktualizácii sa zariadenie reštartuje s novým firmvérom.

Feedback mechanizmus. Po prijatí a vykonaní každého príkazu uzol odošle spätnú väzbu na topic `<príkazový_topic>/feedback` s hodnotou `OK` alebo `ERROR`. Tento mechanizmus umožňuje backendu detekovať výpadky komunikácie a overiť, či bol príkaz skutočne vykonaný.

Globálny STOP. Každý uzol bol prihlásený na topic `<room_id>/STOP`. Pri jeho prijatí sú všetky výstupy okamžite deaktivované bez ohľadu na aktuálny stav uzla. Tento príkaz je kľúčovým bezpečnostným prvkom systému a backend ho odošle automaticky pri ukončení každej scény.

5.1.2 Tlačidlový uzol

Tlačidlový uzol slúži ako fyzický vstupný bod pre spustenie scény. Hardvérové tlačidlo bolo pripojené na GPIO pin 32 cez externý pull-up rezistor (3,3 V), pričom uzol bol nakonfigurovaný v režime `INPUT` bez interného pull-up odporu.

Stlačenie tlačidla prechádza dvojstupňovým filtrovaním. Prvý stupeň je debounce filter s oknom 60 ms, ktorý eliminuje mechanické zákmity kontaktov. Druhý stupeň je cooldown ochrana s lehotou 4 sekúnd, ktorá zabraňuje viacnásobnému spusteniu scény pri rýchlom opakovanom stlačení alebo neúmyselnom podržaní tlačidla. Pokiaľ stlačenie cooldown filtrom neprejde, udalosť sa zaloguje, avšak správa odoslaná nie je.

Po úspešnom prejdení oboch filtrov je správa `START` publikovaná na topic `room1/scene`. Backend na Raspberry Pi je prihlásený na tento topic a spustenie predvolenej scény je ním okamžite iniciované.

5.1.3 Motorický uzol

Motorický uzol bol navrhnutý na riadenie dvoch jednosmerných DC motorov prostredníctvom H-mostíkových ovládačov. Každý motor má priradené dva PWM výstupy (ľavý a pravý smer) a jeden enable pin. PWM signál je generovaný na frekvencii 20 kHz pomocou periférie LEDC mikrokontroléra ESP32, čo zaručuje tichú prevádzku bez počuteľného piskania vinutia.

Uzol prijíma príkazy vo formáte `ON:<rýchlosť>:<smer>[:<rampa>]`, kde rýchlosť je zadaná buď v percentách (0 – 100), alebo priamo ako PWM hodnota (0 – 255), smer hodnotou `L` alebo `R` a voliteľný parameter rampy určuje čas rozbehu v milisekundách. Príkazom `OFF` je motor plynulo zastavený.

Kľúčovou vlastnosťou implementácie je bezpečná zmena smeru za behu. Pokiaľ motor práve beží a je prijatý príkaz na opačný smer, firmvér neprepne smer okamžite — najprv je motor plynulo spomalený na nulu a až potom je smer zmenený a motor rozbehnutý na požadovanú rýchlosť. Tento postup chráni mechaniku pohonu pred opotrebovaním a pred nárazovými prúdmi. Ak je zadaný parameter rampy, rozbeh prebieha lineárnou interpoláciou medzi aktuálnou a cieľovou rýchlosťou v definovanom čase.

Pre každú zmenu stavu motora je spätná väzba odoslaná na topic `room1/motor<N>/feedback`, čo umožňuje backendu sledovať reálny stav pohonov počas scény.

5.1.4 Reléový uzol – spínanie záťaží

Na ovládanie ôsmich výstupných kanálov bol zvolený rozšiřovač I/O portov PCF8574 pripojený cez zbernicu I²C (SDA: GPIO 42, SCL: GPIO 41). Každý kanál zodpovedá jednému efektu expozície: napájaniu dymového stroja, svetlu ohňa, alarmu, dymovému efektu a štyrom dekoratívnym svetlám.

Mapovanie MQTT topicov na fyzické piny bolo centralizované v konfiguračnej štruktúre `DEVICES[]`, kde je pre každé zariadenie definovaný názov topicu, číslo pinu, príznak inverzie výstupu a voliteľný čas automatického vypnutia. Tento prístup umožňuje pridávať alebo presúvať zariadenia bez zásahu do riadiacej logiky uzla.

Pre zariadenia fyzicky obmedzené v maximálnej dobe prevádzky bol implementovaný AutoOff mechanizmus — po uplynutí definovaného času (12 s pre dymový efekt) sa kanál automaticky vypne bez zásahu backendu. Tým sa predchádza poškodeniu zariadenia pri prípadnom výpadku MQTT komunikácie počas scény.

Uzol ďalej podporuje efektové skupiny (`room1/effects/<skupina>`), v rámci ktorých viacero svetiel bliká asynchrónne s náhodne volenými intervalmi zopnutia a rozopnutia. Táto funkcionalita je riadená lokálne na uzle pomocou neblokujúcich časovačov (`millis()`), čím sa minimalizuje závislosť od latencie siete pri vizuálnych efektoch.

Na doske je osadená RGB LED dióda NeoPixel, ktorá poskytuje vizuálnu diagnostiku stavu uzla bez potreby sériového monitora. Zelené pulzovanie (breathing efekt) indikuje plnú prevádzku, oranžové blikanie signalizuje výpadok MQTT a červené blikanie indikuje chýbajúce WiFi pripojenie.

5.2 Backend Raspberry Pi

Backend bol nasadený ako systémová služba `museum-system.service` pod správou `systemd` a bol implementovaný v jazyku Python 3. Architektúra bola rozdelená do piatich funkčných vrstiev, ktoré sú od seba zámerme oddelené: vstupný bod a orchestrácia, stavový automat, mediálna vrstva, komunikačná vrstva MQTT a webové rozhranie, ako je znázornené na obrázku 5.2.

[MIESTO PRE OBRÁZOK: Architektúra backendu Raspberry Pi — päť funkčných vrstiev a ich vzájomné prepojenie (orchestrácia, FSM engine, mediálna vrstva, MQTT vrstva, Web)]

5.2.1 Orchestrátor systému – MuseumController

Trieda `MuseumController` v module `main.py` bola navrhnutá ako orchestrátor celého systému — sama neimplementuje doménovú logiku, ale inicializuje všetky služby, prepája ich callbackmi a riadi hlavnú slučku programu. Štart prebieha v troch fázach: aktivácia bootstrap loggingu, inicializácia komponentov v definovanom poradí pomocou `ServiceContainer` (AudioHandler, VideoHandler, MQTT vrstva, SystemMonitor, ButtonHandler) a wiring callbackov medzi nimi — napríklad `MQTTMessageHandler` dostane referenciu na `SceneParser`, aby mohli byť prijaté MQTT správy preposielané do fronty udalostí stavového automatu. Hlavná slučka následne v pravidelných intervaloch vykonáva health check MQTT spojenia a obsluhu GPIO vstupu. Pri prijatí signálu `SIGTERM` je scéna korektne ukončená, štatistiky sú uložené a všetky výstupy sú zastavené.

5.2.2 Stavový automat a engine scén

Jadro systému tvorí konečný stavový automat (FSM), ktorého návrh bol zdôvodnený v kapitole 3.4. Implementácia bola rozdelená do štyroch vzájomne spolupracujúcich tried podľa princípu jednej zodpovednosti, ako je znázornené na obrázku 5.2.2.

[MIESTO PRE OBRÁZOK: Diagram tried stavového automatu — StateMachine, SceneParser, StateExecutor, TransitionManager a tok dát medzi nimi]

StateMachine slúži na načítanie scény zo súboru JSON a overenie jej správnosti v dvoch krokoch. Prvým krokom je schémová validácia pomocou knižnice `jsonschema`, ktorá overí, že všetky povinné polia sú prítomné a majú správny dátový typ. Druhým krokom je logická validácia — skontroluje sa, že každý cieľový stav uvedený v prechodoch skutočne existuje v slovníku stavov. Táto dvojúrovňová validácia zabraňuje spusteniu chybne nakonfigurovanej scény. Po úspešnom načítaní trieda udržuje informácie o aktuálnom stave a čase vstupu do neho.

SceneParser koordinuje celý životný cyklus scény od načítania až po ukončenie. Pred spustením scény je JSON prehľadaný a identifikované sú všetky zvukové súbory s prefixom `sfx_`, ktoré sú následne prednahrané do RAM pamäte (*preload*), čím je eliminovaná latencia pri prehrávaní zvukových efektov počas behu scény. V každej iterácii hlavného cyklu je vykonaný nasledovný sled operácií: overenie, či dobehlo audio alebo video a prípadná registrácia udalosti, vyhodnotenie globálnych prechodov platných pre celú scénu, vykonanie timeline akcií aktuálneho stavu s uplynulým časom a nakoniec vyhodnotenie lokálnych prechodov.

StateExecutor má na starosti vykonávanie akcií definovaných v stavoch scény. Bola implementovaná dispatch tabuľka, kde kľúčom je typ akcie a hodnotou príslušná obslužná metóda. Systém podporuje tri typy akcií:

- `mqtt` — správa je odoslaná na zadaný topic MQTT brokera, napríklad `{"action":"mqtt","topic'`
- `audio` — príkaz je odoslaný audio handleru, napríklad `{"action":"audio","message":"PLAY:sf`
- `video` — príkaz je odoslaný video handleru, napríklad `{"action":"video","message":"PLAY_VI`

Dispatch tabuľka umožňuje pridávanie nových typov akcií bez modifikácie existujúceho kódu. Zároveň je sledované, ktoré timeline akcie boli v rámci aktuálneho stavu už vykonané, aby nedochádzalo k ich opakovanému spusteniu pri každej iterácii cyklu.

TransitionManager má na starosti vyhodnocovanie podmienok prechodov medzi stavmi. Spravuje tri thread-safe fronty udalostí: pre MQTT správy, pre ukončenie audio súborov a pre ukončenie videa. Bolo implementovaných päť typov prechodov:

- `timeout` — prechod nastane po uplynutí definovaného počtu sekúnd od vstupu do stavu, napríklad `{"type":"timeout","delay":4,"goto":"HODINY_TIKANIE_1"}`,
- `audioEnd` — prechod nastane po prirodzenom ukončení prehrávania zadaného audio súboru,
- `videoEnd` — prechod nastane po prirodzenom ukončení prehrávania videa,
- `mqttMessage` — prechod nastane pri príchode konkrétnej správy na konkrétny topic, napríklad `{"type":"mqttMessage","topic":"room1/emergency","message":"STOP","goto":`
- `always` — okamžitý nepodmienený prechod, typicky používaný na reťazenie stavov bez čakacej podmienky.

Okrem lokálnych prechodov jednotlivých stavov môže scéna definovať **globálne udalosti** (`globalEvents`), ktoré sú vyhodnocované v každej iterácii bez ohľadu na to, v ktorom stave sa scéna práve nachádza. Typicky boli definované dva globálne prechody: núdzové zastavenie scény pri prijatí správy `STOP` na topic `room1/emergency` a maximálny časový limit scény ako ochrana pred nekonečným čakaním pri prípadnom výpadku hardvérového zariadenia. V nasadenej scéne `SceneV01.json` bol globálny timeout nastavený na 600 s.

5.2.3 Mediálna vrstva – audio a video

Pre prehrávanie zvuku bola implementovaná trieda `AudioHandler`, ktorá obaľuje knižnicu `pygame.mixer` a implementuje dve paralelné cesty prehrávania. Zvukové efekty (prefix `sfx_`) boli vopred načítané do RAM ako objekty `Sound` a prehrávajú sa priamo z pamäte na jednom z 32 mixérových kanálov, čo umožňuje polyfóniu — súčasné prehrávanie viacerých zvukov bez vzájomného blokovania. Dlhšie nahrávky (atmosféra, hudba na pozadí) sú streamované z disku cez `pygame.mixer.music`, čím sa šetrí RAM. Po prirodzenom ukončení prehrávania je `TransitionManager` notifikovaný, čím sú umožnené prechody typu `audioEnd`.

Pre prehrávanie videa bol zvolený proces `mpv` riadený cez Unix IPC socket (`/tmp/mpv_socket`). V kľudovom stave je zobrazovaný čierny obrázok (`black.png`), čím je eliminované blikanie obrazovky pri prechodoch medzi videom a stavom bez videa. V pravidelných intervaloch je overované zdravie procesu a IPC odozva; pri zlyhaní je vykonaný pokus o automatický reštart s konfigurovateľným počtom opakovaní a čakacou dobou.

5.2.4 Komunikačná vrstva MQTT

MQTT vrstva bola rozdelená do štyroch tried, ktorých úlohy nadväzujú na komunikačný model navrhnutý v kapitole 3.5. `MQTTClient` obaľuje knižnicu `paho-mqtt` a zabezpečuje retry logiku pri prvotnom pripájaní a automatické obnovenie odberu topicov po reconnecte. `MQTTMessageHandler` smeruje prichádzajúce správy v prioritnom poradí: stav zariadenia, spätná väzba, štart scény a nakoniec udalosti pre FSM. `MQTTFeedbackTracker` sleduje každý odoslaný príkaz a pri vypršaní časového limitu (1 s) zaloguje chybu s identifikáciou zariadenia bez blokovania behu scény. `MQTTDeviceRegistry` udržuje zoznam aktívnych

zariadení; tie, od ktorých neprišla správa dlhšie ako 25 s, sú automaticky označené ako offline.

5.2.5 Webový dashboard

Webové rozhranie bolo implementované pomocou frameworkov Flask a Flask-SocketIO s architektúrou Blueprintov a prístupom chráneným HTTP Basic Authentication. Kľúčovou vlastnosťou je live streaming logov cez SocketIO — každý záznam je okamžite doručený všetkým pripojeným klientom bez nutnosti obnovovania stránky. Rozhranie umožňuje spúšťanie a zastavovanie scén, manuálne odosielanie MQTT správ, správu mediálnych súborov a vzdialený reštart služby alebo celého Raspberry Pi.

5.3 Konfigurácia scén – formát JSON

Scény boli definované v súboroch JSON uložených v adresári `scenes/<room_id>/`. Každý súbor obsahuje identifikátor scény, počiatočný stav, voliteľné globálne udalosti a slovník stavov. Každý stav môže definovať akcie pri vstupe (`onEnter`), pri odchode (`onExit`), časovo naplánované akcie (`timeline`) a podmienky prechodov (`transitions`). Voľba tohto formátu bola odôvodnená v kapitole 3.5, kde bola editovateľnosť konfigurácie bez zásahu do kódu definovaná ako kľúčová požiadavka systému.

Ako príklad slúži scéna `SceneV01.json` (*Príbeh Oheň*) nasadená v pilotnej expozícii. Scéna obsahuje 11 stavov a realizuje nasledovnú dramaturgiu: atmosférický zvuk s predohrevom dymového stroja, vzplanutie ohňa so svetelným efektom, alarmové blikanie, vypustenie dymu, zvuk vlaku, klikanie a tikanie hodín s odbitím a finálny pohyb hodinového mechanizmu DC motorom. Maximálna dĺžka scény bola ohraničená globálnym timeoutom 600 s. Diagram stavov scény je znázornený na obrázku 5.3.

[MIESTO PRE OBRÁZOK: Diagram stavov scény Príbeh Oheň (SceneV01.json) — 11 stavov so šípkami prechodov a typom každého prechodu (timeout / audioEnd / mqttMessage)]

Formát JSON sa ukázal ako dostatočne expresívny pre zložité multimediálne scénáre s kombináciou audio, video, svetelných a pohybových efektov, pričom zostáva editovateľný priamo cez webové rozhranie systému bez potreby znalosti programovania.

6 Experimentálne overenie

Táto kapitola prezentuje reálne skúšky navrhnutého systému, ktoré overujú splnenie stanovených cieľov zadania v podmienkach reálnej prevádzky.

6.1 Popis fyzického testovania a testovacieho prostredia

Systém bol nasadený a overený na reálnej fyzickej inštalácii **CHRONOS** – steampunkovej expozičnej stene s rozmermi približne 3 m × 1,5 m. Inštalácia integruje dekoratívne hodinové mechanizmy, sústavu ozubených kolies, analógové budíky, dymostroj a scénické osvetlenie, ktorých mechanická konštrukcia je popísaná v kapitole 4.3.

Do testovania boli zapojené nasledujúce uzly a zariadenia:

- **Raspberry Pi** s bežiacim centrálnym procesom `MuseumController`, pripojeným HDMI monitorom a audio výstupom cez 3,5 mm jack.
- **ESP32 Motorický uzol** riadiaci dva jednosmerné DC motory cez H-mostíky BTS7960 – motor hodinového mechanizmu a motor ozubených kolies.
- **ESP32 Reléový uzol** (Waveshare ESP32-S3) spínajúci osem kanálov: napájanie dymostroja (230 V), produkciu dymu (auto-off 12 s), svetlo ohňa, Edisonovu žiarovku a štyri kanály podsvietenia LED pásov (12 V).
- **ESP32 Tlačidlový uzol** ako fyzický štartovací vstup pre návštevníka.

6.2 Testované scenáre a výsledky

Overenie stability systému prebiehalo v troch izolovaných rovinách: prevádzka s hardvérom, softvérová výdrž jadra a odolnosť voči sieťovým výpadkom.

6.2.1 Scenár 1 – Komplexná lineárna scéna s hardvérom

- **Cieľ:** Otestovať plynulosť a asynchrónnu presnosť prechodov definovaných v JSON súbore počas riadenia fyzických aktuátorov priamo v múzeu.
- **Priebeh:** Systém musel prehrať multimediálny audiovizuálny obsah súčasne s presne načasovaným pohybom motorov a dynamickým zhasínaním svetiel.
- **Výsledok:** Všetky akcie boli spustené v časoch definovaných v *timeline*. Komunikácia s mikrokontrolérmi prebehla bez zaznamenaného oneskorenia a synchronizácia audio-video stopy so svetelnými efektmi bola vizuálne konzistentná počas celého trvania testu.

6.2.2 Scenár 2 – Dlhodobý test stability riadiaceho systému

- **Cieľ:** Overiť odolnosť Raspberry Pi a MQTT brokera proti pretečeniu pamäte a zlyhaniu pri dlhodobom plynulom behu (tzv. *Soak test*).
- **Priebeh:** Centrálny program systému bežal na stole formou *dry-run* testovania v kuse niekoľko dní v laboratórnych podmienkach. Priebežne dochádzalo k ojedinelému spúšťaniu scén a monitorovaniu odosielaných MQTT príkazov na zbernicu.
- **Výsledok:** Proces `MuseumController` zostal počas celej doby testu bez reštartu. Príkazy boli generované na MQTT zbernicu priebežne bez výpadku, čo preukázalo dostatočnú stabilitu softvéru pre nepretržitú prevádzku.

6.2.3 Scenár 3 – Test sieťovej odolnosti a LWT (Last Will)

- **Cieľ:** Overiť detekčný a oznamovací mechanizmus systému pri neštandardnej strate bezdrôtového spojenia uzlov na Wi-Fi sieti.
- **Priebeh:** Koncovým ESP32 uzlom (*Controller_MOTORS* a *Controller_RELAY*) bolo striedavo fyzicky odpájané a následne vracané napájanie za behu celého systému.
- **Výsledok:** Po obnovení napájania sa uzly autonómne pripojili späť na Wi-Fi sieť a MQTT brokera. Dashboard správne zaznamenal zmenu stavu - prepis indikátora z `offline` na `online` prebehol na základe LWT správy od brokera a následného `online` hlásenia od uzla.

6.2.4 Scenár 4 – Meranie latencie MQTT spätnej väzby

- **Cieľ:** Kvantitatívne overiť latenciu cyklu príkaz – spätná väzba medzi backendom Raspberry Pi a motorickým uzlom ESP32 pri vysokofrekvenčnom zaťažení (10 príkazov za sekundu, stres-test TC-08).
- **Priebeh:** Počas behu stres-testovej scény bolo z databázového záznamu modulu `mqtt_feedback` extrahovaných 128 hodnôt doby odozvy – po 64 pre každý motorický kanál. Meranie pokrývalo súvislý časový úsek 40 sekúnd.
- **Výsledok:** Priemerná latencia celého cyklu dosiahla 88,7 ms pri mediáne 90 ms. Žiadna zo 128 meraných hodnôt neprekročila hranicu 200 ms – 59,4 % meraní spadalo do pásma pod 100 ms a zvyšných 40,6 % do pásma 100 – 200 ms (Tab. 6.1, Obr. 6.1). Hodnota P99 dosiahla 166 ms, čo potvrdzuje absenciu extrémnych odchýlok. Kanál `motor2` vykazoval v priemere o 17,9 ms vyššiu latenciu ako `motor1`; rozdiel je zhodný s typickým variabilným oneskorením domácej Wi-Fi siete a nemá vplyv na pozorovateľnosť efektov.

Tabuľka 6.1: Štatistika latencie MQTT cyklu príkaz – spätná väzba

Zariadenie	n	Priemer (ms)	Medián (ms)	Min (ms)	Max (ms)	P95 (ms)
motor1	64	79,7	81,0	19	143	136
motor2	64	97,6	102,0	17	167	164
Celkovo	**128**	**88,7**	**90,0**	**17**	**167**	**156**

Obrázok 6.1: Distribúcia latencie MQTT spätnej väzby (n = 128, stres-test TC-08, 10 Hz)

6.3 Zhodnotenie overenia

Všetky tri testované scenáre boli úspešne overené. Architektúra distribuovanej hviezdicovej siete sa ukázala ako dostatočne stabilná pre podmienky plného fyzického zaťaženia v prostredí expozície. Bezdrôtová komunikácia cez protokol MQTT preukázala reaktivitu postačujúcu na spracovanie hardvérových triggerov bez merateľného oneskorenia, čo vylúčilo potrebu prechodu na drôtové prepojenie.

7 Záver

Cieľom práce bol návrh a realizácia modulárneho riadiaceho systému pre interaktívne múzejné expozície, ktorý by umožnil synchrónne ovládanie fyzických efektov a multimediálneho obsahu z jediného centrálneho uzla, pričom priebeh expozície mal byť konfigurovateľný bez zásahu do zdrojového kódu. Všetky štyri ciele zadania boli naplnené a systém bol úspešne nasadený a overený v podmienkach reálnej fyzickej inštalácie.

Hlavným inžinierskym prínosom práce je preukázanie, že kombinácia dostupných IoT platforiem s architektúrou konečného stavového automatu (FSM) a textovou konfiguráciou scenárov predstavuje funkčnú alternatívu ku komerčným show-control riešeniam (viď kapitola 2.3.2) aj priemyselným PLC systémom (viď kapitola 2.3.1). Navrhnutý systém poskytuje porovnateľnú funkcionalitu pri výrazne nižšej vstupnej bariére – nelineárne vetvenie scenára, synchronizáciu médií a riadenie výkonových prvkov je možné dosiahnuť na otvorených platformách bez proprietárneho softvéru. Oproti jednoduchým sekvenčným riešeniam je kľúčovým rozdielom schopnosť systému čakať na vstup od návštevníka a podľa neho dynamicky rozvetvovať priebeh expozície, pričom ostatné podmienky prechodu sú vyhodnocované súčasne a neblokujúco.

Z hardvérového hľadiska (kapitola 4) bola architektonická voľba postavená na uprednostnení hotových, preverených priemyselných modulov pred návrhom vlastných výkonových obvodov. Tento prístup sa v prevádzke osvedčil: systém pri fyzickom testovaní nevykázal žiadne výpadky spôsobené hardvérovým zlyhaním a prípadná výmena komponentu pri poruche nevyžaduje špeciálne znalosti elektroniky.

Softvérová architektúra (kapitola 5) dosiahla kľúčovú vlastnosť, ktorá priamo napĺňa požiadavku na modularitu definovanú v kapitole 3.1: celá choreografia expozície je uložená výhradne v externých konfiguračných súboroch, oddelených od kódu enginu. Obsluha múzea tak môže meniť časovanie, poradie efektov aj priebeh scenára bez akéhokoľvek zásahu do zdrojového kódu – čo je z pohľadu dlhodobej prevádzkovateľnosti inštalácie zásadná vlastnosť.

Experimentálne overenie (kapitola 6) potvrdilo stabilitu systému v troch rovinách: presnosť časovania efektov počas reálnej scény s fyzickým hardvérom, odolnosť centrálneho procesu pri viacdňovom záťažovom teste bez reštartu a správne automatické zotavenie siete po výpadku napájania uzlov bez zásahu obsluhy.

Systém má dve identifikované prevádzkové obmedzenia. Bezdrôtová komunikácia kladie na múzejnú prevádzku požiadavku oddelenej siete; ak je tá istá sieť zdieľaná s verejnosťou, môže to negatívne ovplyvniť reaktivitu systému. Druhým obmedzením je počiatočná konfigurácia každého nového uzla, ktorá si pred prvým nasadením vyžaduje fyzický prístup k zariadeniu. Po nasadení sú však všetky následné aktualizácie firmvéru riešiteľné bezdrôtovo na diaľku.

Do budúcna sa systém prirodzene otvára dvom smerom rozvoja. Prvým je rozšírenie webového dashboardu o vizuálny editor scenárov, ktorý by obsluhe umožnil skladať priebeh expozície grafickým presúvaním stavov bez manuálneho editovania textových sú-

borov. Druhým je podpora väčšieho počtu miestností, keďže architektúra systému bola od začiatku navrhovaná s ohľadom na toto rozšírenie (viď kapitola 3).

Zoznam obrázkov

2.1 Centralizovaná topológia 11
2.2 Distribuovaná topológia 11

3.1 Fyzická topológia riadiaceho systému 18
3.2 Diagram Konečného stavového automatu 19
3.3 Sekvenčný diagram toku MQTT správ 20

6.1 Distribúcia latencie MQTT spätnej väzby (n = 128, stres-test TC-08, 10 Hz) 33

Zoznam tabuliek

2.1 Úrovne kvality služby (QoS) v MQTT 13

6.1 Štatistika latencie MQTT cyklu príkaz – spätná väzba 32

Použitá literatúra

[1] IEC 61131-3:2013. *Programmable Controllers – Part 3: Programming Languages* [online]. Geneva: IEC, 2013 [cit. 2026-03-15]. Dostupné z: https://webstore.iec.ch/en/publication/4552

[2] JOHN, K. H. a TIEGELKAMP, M. *IEC 61131-3: Programming Industrial Automation Systems*. 2. vyd. Berlin: Springer, 2010. ISBN 978-3-642-12014-5. Dostupné z: https://link.springer.com/book/10.1007/978-3-642-12015-2

[3] STILLWELL, B. Putting the "Play"Back into Display: Interactive Exhibits in Small Museums [online]. Eugene: University of Oregon, 2017 [cit. 2026-03-15]. Dostupné z: https://scholarsbank.uoregon.edu/server/api/core/bitstreams/810e909e-35d7-4255-8119-dbcd4cc16104/content

[4] Research on the Strategy and Practical Mechanism of Museum Exhibition Space Design Based on Immersive Experience. In: *Proceedings of ETMHS 2025* [online]. 2025 [cit. 2026-03-15]. Dostupné z: https://webofproceedings.org/proceedings_series/ESSP/ETMHS%202025/H27.pdf

[5] Computational Frameworks for Interactive Multimedia Art Installations in Public Spaces using Real Time Digital Control and Visualization. *PeerJ Computer Science* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://peerj.com/articles/cs-3384.pdf

[6] TKÁČIK, M. a kol. Modeling and Analysis of Distributed Control Systems: Proposal of a Methodology. *Processes* [online]. 2024, roč. 12, č. 1 [cit. 2026-03-15]. Dostupné z: https://www.mdpi.com/2227-9717/12/1/5

[7] SHEHADE, M. a STYLIANOU-LAMBERT, T. Virtual Reality in Museums: Exploring the Experiences of Museum Professionals. *Applied Sciences* [online]. 2020, roč. 10, č. 11, článok 4031 [cit. 2026-03-20]. Dostupné z: https://www.mdpi.com/2076-3417/10/11/4031

[8] BACHI, V. a kol. Museums and the Post-Digital: Revisiting Challenges in the Digital Transformation of Museums. *Heritage* [online]. 2024, roč. 7, č. 2, s. 890–910 [cit. 2026-03-20]. Dostupné z: https://www.mdpi.com/2571-9408/7/2/41

[9] ZONTA, D. a kol. Real-time health monitoring of historic buildings with wireless sensor networks. *Smart Structures and Systems* [online]. 2010, roč. 6, č. 5, s. 595–615 [cit. 2026-03-15]. Dostupné z: https://shm.cs.illinois.edu/Full%20Scale%20Papers/14.%20Zonta%20et%20al.%202010.pdf

[10] Design and Implementation of Intelligent Audio-Video Interaction and Control Platform Based on Distributed Architecture. In: *MATEC Web of Conferences* [online]. 2021 [cit. 2026-03-15]. Dostupné z: https://www.matec-conferences.org/articles/matecconf/pdf/2021/05/matecconf_cscns20_05002.pdf

[11] AL-SHUHAIB, T. Low-Cost ESP32, Raspberry Pi, Node-Red, and MQTT Protocol Based SCADA System. *IEEE International Conference on Intelligent Systems*. 2020. Dostupné z: https://www.researchgate.net/publication/347154219

[12] What is Medialon? Competitors, Complementary Techs & Usage. *Sumble* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://sumble.com/tech/medialon

[13] Alcorn McBride – Reliable AV Technology for Themed Attractions [online]. 2026 [cit. 2026-03-15]. Dostupné z: https://alcorn.com/

[14] Engineering the Invisible Magic: Alcorn McBride at 40. *InPark Magazine* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://www.inparkmagazine.com/109alcornmcbride/

[15] HUNTINGTON, John. *Show Networks and Control Systems*. 2. vyd. Campbell, CA: Zircon Designs Press, 2017. ISBN 978-1138090874.

[16] NOURAL, A., ATIQUZZAMAN, M. a GAEDKE, M. Software-defined networking: architecture, protocols, and applications. *Journal of Information and Communication Technology* [online]. 2019, roč. 15, s. 796–810 [cit. 2026-03-15]. Dostupné z: https://www.emerald.com/ftnet/article/15/1-3/1/1334816/Software-defined-networking-architecture-protocols

[17] PATITUCCI, A. a kol. Interactive Museum Exhibits with Microcontrollers: A Use-Case Scenario. *arXiv* [online]. 2015 [cit. 2026-03-15]. Dostupné z: https://arxiv.org/abs/1509.07626

[18] AMIRKHANOV, B. a kol. Evaluating HTTP, MQTT over TCP and MQTT over WEBSOCKET for Digital Twin Applications: A Comparative Analysis on Latency. *IJIRSS* [online]. 2025 [cit. 2026-03-15]. Dostupné z: https://www.ijirss.com/index.php/ijirss/article/download/4414/623/7222

[19] YOKOTANI, T. a SASAKI, Y. Comparison with HTTP and MQTT on required network resources for IoT. In: *2016 International Conference on Control, Electronics, Renewable Energy and Communications (ICCEREC)* [online]. IEEE, 2016 [cit. 2026-03-15]. Dostupné z: https://ieeexplore.ieee.org/document/7814989

[20] Comparative Analysis of Power Consumption between MQTT and HTTP Protocols in an IoT Platform Designed for Remote Real-Time Monitoring. *PMC – PubMed Central* [online]. 2023 [cit. 2026-03-15]. Dostupné z: https://pmc.ncbi.nlm.nih.gov/articles/PMC10224120/

[21] TSAQIEF, M. F. a SUTOPO, J. Comparative Performance Analysis Between the MQTT and WebSocket Protocols. *bit-Tech* [online]. 2025 [cit. 2026-03-15]. Dostupné z: https://jurnal.kdi.or.id/index.php/bt/article/download/3223/1701/20910

[22] ISO/IEC 20922:2016. *Information Technology – Message Queuing Telemetry Transport (MQTT) v3.1.1* [online]. Geneva: IEC, 2016 [cit. 2026-03-15]. Dostupné z: https://www.iso.org/standard/69466.html

[23] The Use of MQTT in M2M and IoT Systems: A Survey. *ResearchGate* [online]. 2020 [cit. 2026-03-15]. Dostupné z: https://www.researchgate.net/publication/346414150_The_Use_of_MQTT_in_M2M_and_IoT_Systems_A_Survey

[24] MQTT Protocol for the IoT – Review Paper. *ResearchGate* [online]. 2023 [cit. 2026-03-15]. Dostupné z: https://www.researchgate.net/publication/373640610_MQTT_Protocol_for_the_IoT_-_Review_Paper

[25] Raspberry Pi 4 Model B – Datasheet. *Raspberry Pi Ltd* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://datasheets.raspberrypi.com/rpi4/raspberry-pi-4-datasheet.pdf

[26] MAIER, A., SHARP, A. a VAGAPOV, Y. Comparative analysis and practical implementation of the ESP32 microcontroller module for the Internet of Things. In: *Proceedings of the 7th International Conference on Internet Technologies and Applications (ITA 2017)* [online]. IEEE, 2017, s. 143–148 [cit. 2026-03-15]. Dostupné z: https://ieeexplore.ieee.org/document/8101926

[27] AZAD, T. a kol. IoT edge network interoperability. *Computer Communications* [online]. 2025, roč. 236, s. 108125 [cit. 2026-03-15]. Dostupné z: https://doi.org/10.1016/j.comcom.2025.108125

[28] Darlington Transistor vs. MOSFET: Selection Guidelines. *Cadence PCB Design & Analysis* [online]. 2023 [cit. 2026-03-15]. Dostupné z: https://resources.pcb.cadence.com/blog/2023-pdarlington-transistor-vs-mosfet-selection-guidelines

[29] MOSFET H-Bridge vs Dedicated Motor Driver Solutions. *ALLPCB* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://www.allpcb.com/allelectrohub/mosfet-h-bridge-vs-dedicated-motor-driver-solutions

[30] Solid State Relay Guide. *Phidgets Support* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://www.phidgets.com/docs/Solid_State_Relay_Guide

[31] Zero-Cross or Random Relay: What are the Differences? *Celduc Relais* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://www.celduc-relais.com/en/zero-cross-or-random-relay-what-are-the-differences/

[32] Beware of Zero-Crossover Switching of Transformers. *TE Connectivity* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://www.te.com/en/products/relays-and-contactors/electromechanical-relays/intersection/zero-crossover-switching-transformers.html

[33] KAUR, A. a kol. A Literature Review on Device-to-Device Data Exchange Formats for IoT Applications. *Journal of Intelligent Systems and Computing* [online]. 2020 [cit. 2026-03-15]. Dostupné z: https://www.researchgate.net/

publication/349022302_A_Literature_Review_on_Device-to-Device_Data_Exchange_Formats_for_IoT_Applications

[34] YAML vs. JSON: Definitions, Pros & Cons. *Built In* [online]. 2024 [cit. 2026-03-15]. Dostupné z: https://builtin.com/software-engineering-perspectives/yaml-vs-json

[35] CELERDATA. YAML, JSON and XML: A Practical Guide to Choosing the Right Format. *CelerData Blog* [online]. 2024 [cit. 2025-02-20]. Dostupné z: https://celerdata.com/glossary/yaml-json-and-xml-a-practical-guide-to-choosing-the-right-format

[36] BLANCHON, B. How to Reduce Memory Usage? – ArduinoJson 7 [online]. 2026 [cit. 2026-03-15]. Dostupné z: https://arduinojson.org/v7/how-to/reduce-memory-usage/

[37] PARTARAKIS, N., ANTONA, M. a STEPHANIDIS, C. Adaptable, personalizable and multi-user museum exhibits. In: *Curating the Digital*. Cham: Springer, 2016, s. 167–179. ISBN 978-3-319-13005-9.